

Tensor4all stack: development update

tenferro-rs · tensor4all-rs · hataori-rs · bindings

Hiroshi Shinaoka

tensor4all meeting, 2026-09-22

With Ryo Watanabe, Satoshi Terasaki, Marc K. Ritter (ACI / TCI algorithms), Selina Dirnböck, Markus Frankenbach, Nepomuk Ritz, Jin-Guo Liu

The stack, and what this talk covers

Layer	Repository and role
Bindings	Tensor4all.jl (Julia via C API) · tensor4all-py (PyO3 prototype)
Tensor networks	tensor4all-rs : TreeTN, TCI, ACI, quantics, partitioned networks
Dense tensor engine	tenferro-rs : tensors, sessions, einsum, linalg, FFT, autodiff; CPU / CUDA / WebGPU / Metal
Parallel runtime	hataori-rs : dynamically scheduled <code>pmap</code> over threads and MPI ranks

- Period: mid-May to mid-September 2026. Full features live in Rust; Julia and Python are thin entry points.
- Plan: tenferro-rs (20 min) → tensor4all-rs (20 min) → hataori-rs, bindings, next steps (15 min).

Four months at a glance

Repository	Merged PRs	Releases
tenferro-rs	about 350	v0.2 (Jul) → v0.3 → v0.4 → v0.5 (Sep 8) , on crates.io
tensor4all-rs	about 115	v0.2.0 (Jun)
strided-rs	about 65	v0.3 (Jul) → v0.4 (Aug), on crates.io
hataori-rs	new	first working engine (Aug)
Tensor4all.jl	10	Windows CI

- Every release ships migration notes; GPU CI runs on RunPod.
- Review, not typing, is the bottleneck: most code is written by agents under human gates (last slides).

tenferro-rs: sessions instead of per-call setup

tenferro-rs

- **Finding:** on small tensors the gap to PyTorch was per-operation overhead, not the kernels.
- **Answer:** one session API shared by eager, autodiff and traced execution. Buffer pools, Rayon pool and BLAS handles live in the session.
- tensor4all-rs now runs on an explicit CPU / CUDA execution context.

```
let cpu = CpuBackend::new()?;           // build once, reuse everywhere
let s = cpu.session();                  // enter a shared execution scope
let c = einsum_in(&s, "ij,jk->ik", [&a, &b])?;
```

- **Linear algebra:** full and compact SVD, incremental Householder QR, rank-revealing QR; derivative rules generated from linearization, with opt-in gauge conventions.
- **Einsum:** strided-rs replay plans compiled once, ellipsis everywhere, TBLIS and grouped GEMM providers, a tropical (semiring) extension.
- **CUDA:** cached cuTENSOR plans, cuFFT, on-device QR, asynchronous event contract. Unsupported ops return typed errors, never a silent CPU fallback.
- **Portable:** WebGPU and Metal through CubeCL / CubeK, still experimental.

Benchmark: CPU einsum, Apple M5

Max

tenferro-benchmark

Instance (ms)	tenferro trace	tenferro eager	PyTorch	JAX	OMEinsum
matmul 1024	8.4	5.3	5.2	35.5	40.7
MERA closed network	177	190	160	848	980
batched likelihood (LM)	10.4	26.1	18.4	29.7	85.8
tensor-network permutation	123	156	134	212	180

- Same BLAS (Accelerate), one thread, medians, refreshed mid-September.
- Traced mode matches PyTorch on tensor-network-shaped instances; eager mode still pays dispatch on tiny ops.

- **New algorithms:** randomized TreeTN contraction, cross interpolation native to trees, partitioned tree networks with reconstruction.
- **Parallel:** adaptive interpolation runs on hataori-rs, Rayon by default and MPI opt-in.
- **Quality:** an August audit removed library panics, introduced typed errors, layering rules and a performance evidence ledger.
- **Boundaries:** C API for Julia unchanged; a PyO3 crate for Python started.

TreeTN: ground states, dynamics, storage tensor4all-rs

- Two-site DMRG, TDVP and GSE-TDVP on general trees; data structures and sweep plans follow ITensorNetworks.jl.
- **Root-edge-first sweep ordering:** fixed a ten-orders-of-magnitude accuracy loss on branching trees.
- Variational fitting of sums of TreeTNs; dense tensor $\rightarrow$ TT-SVD constructor.
- HDF5 schema for TreeTNs and an explicit CUDA contraction path.

Successive randomized compression (SRC)

tensor4all-rs

- Randomized algorithm for the compressed $\text{MPO} \times \text{MPS}$ product, by Camaño, Epperly and Tropp; extended here to general trees.
- Fixed-rank and adaptive variants, Rust and C API. Adaptive SRC rests on tenferro's incremental Householder QR and rank-revealing QR.
- CUDA-resident TreeTNs stay on device through the whole contraction.
- Verified line by line against the authors' reference code; the provenance audit is in the repository docs.

TreeACI: alternating cross interpolation on trees

tensor4all-rs

- ACI approximates elementwise functions of tensor trains by cross interpolation (Ritter et al.). Our chain crate is a port of `AlternatingCrossInterpolation.jl`.
- **New:** ACI runs on a `TreeTN` directly, same options as the chain version, n -way products in one run.
- Sweep is a deterministic minimum-retracing walk of length $2|E|$ — diameter, reducing to the usual two sweeps on a chain.
- Chain ACI gained a global-pivot guard and cache reuse across sweeps.

- **PartitionedTreeTN**: adaptive patching (Grosso et al.) generalized from tensor trains to arbitrary tree topologies.
- Patch error budgets proportional to patch volume, $\varepsilon_p^2 \propto \text{vol}_p$.
- **Reconstruction**: repartition a patched target under a global L2 allowance; operators on index subsets such as the quantics Fourier transform.
- **Parallel adaptive interpolation**: patch waves scheduled by hataori-rs, deterministic sampling across serial, Rayon and MPI runs.

hataori-rs: tasks over threads and ranks

hataori-rs

- A Rust engine for a dynamically scheduled `pmap`, inspired by `Distributed.jl`. Task granularity: seconds to minutes.
- One API, four modes chosen by Cargo features: serial, Rayon, MPI, hybrid (ranks × threads).
- MPI through `rsmpi` or a runtime-loaded MPI ABI, so it shares the runtime of `MPI.jl` and `mpi4py`.
- Used by `tensor4all-rs` adaptive interpolation and a `tenferro` adapter. A long-lived distributed runtime is under design.

Tensor4all.jl (best effort)

- For light use and teaching; heavy users call tensor4all-rs directly
- Windows CI, RustToolChain update, QR tolerance through the FFI
- Usage audit skill: 12 downstream failure modes; batched readout beat pointwise readout by three orders of magnitude

tensor4all-py (PyO3 prototype)

- Direct PyO3 boundary, not through the C API
- Index, Tensor, TreeTN, batched TreeTCI and quantics cross interpolation from NumPy callbacks
- Not yet: workspace membership, CI, wheels

How we develop, and what comes next

Process

- Issues carry acceptance criteria; agents implement, read-only reviewer models gate design and diff
- **Profile first:** no optimization without a paired measurement
- Provenance policy and CITATION.cff per repository

Next

- tenferro-rs JOSS paper, mid-October, with Jin-Guo Liu's Rust quantum circuit simulator
- SRC and TreeACI benchmarks against zip-up and fitting on quantics workloads
- Python: CI, wheels; Tensor4all.jl: catch up to TreeTN

Discussion: which of these do you need first?

References

1. Y. Núñez Fernández, M. K. Ritter, et al., Learning tensor networks with tensor cross interpolation: new algorithms and libraries, *SciPost Phys.* 18, 104 (2025).
2. Y. Núñez Fernández et al., Learning Feynman diagrams with tensor trains, *Phys. Rev. X* 12, 041018 (2022).
3. M. K. Ritter, Y. Núñez Fernández, M. Wallerberger, J. von Delft, H. Shinaoka, X. Waintal, Quantics tensor cross interpolation, *Phys. Rev. Lett.* 132, 056501 (2024).
4. M. K. Ritter et al., Alternating cross interpolation, *arXiv:2604.00037* (2026); *AlternatingCrossInterpolation.jl*.
5. C. Camaño, E. N. Epperly, J. A. Tropp, Successive randomized compression, *arXiv:2504.06475* (2025); *RandomMPOMPS*.
6. G. Grosso, M. K. Ritter, et al., Adaptive patching for tensor train computations, *arXiv:2602.22372* (2026).
7. M. Fishman, S. R. White, E. M. Stoudenmire, The ITensor software library, *SciPost Phys. Codebases* 4 (2022); *ITensorNetworks.jl*.
8. S. R. White, *Phys. Rev. Lett.* 69, 2863 (1992); J. Haegeman et al., *Phys. Rev. B* 94, 165116 (2016); M. Yang and S. R. White, *Phys. Rev. B* 102, 094315 (2020).
9. J. Chen and M. Lindsey, Direct interpolative construction of the DFT as an MPO, *arXiv:2404.03182*.
10. Software: github.com/tensor4all/tenferro-rs, *tensor4all-rs*, *tenferro-benchmark*, *Tensor4all.jl*; github.com/shinaoka/hataori-rs.